

Arquitectura del sistema

Censo Municipal de Animales · Navalcarnero

Arquitectura

Visión general de los componentes del sistema y cómo se relacionan.

Diagrama lógico



Componentes

Frontend (web/)

- SPA de una sola página (`web/index.html`) con includes SSI que componen el

HTML en tiempo de servir. Cada vista vive en `web/pages/`.

- JavaScript vanilla en `web/js/`, organizado por dominio

(`animales.js`, `propietarios.js`, `policia.js`, `auditoria.js`, `backups.js`, etc.).

- Estilos en `web/css/` (`base.css`, `forms.css`, `pages.css`, `print.css`).
- PWA con `manifest.json` y `sw.js`.
- Las imágenes subidas se sirven como ficheros estáticos desde

`web/fotos_animales/` y `web/fotos_incidentes/`.

Servidor web

- **Producción (Docker):** `nginx` sirve estáticos y hace proxy a la API.

Define dos vhosts: el público (puerto 80, X-Admin-Access vacío) y el de administración (puerto 8181, X-Admin-Access = true).

- **Local sin Docker:** `run_local.py` levanta dos `http.server` en los

mismos puertos, replica el SSI manualmente y hace de proxy a Flask.

API (`api/app.py`)

- Flask + `mysql.connector` + `bcrypt` + `apscheduler`.
- Lógica de negocio, autenticación por token, auditoría JSONL,

recordatorios automáticos y backups programados.

- Auto-inicializa tablas auxiliares (`INCIDENCIAS`, `EMPLEADOS`,

`POLICIA_USUARIOS`, etc.) si no existen al arrancar.

Base de datos (`db/`)

- MariaDB 10.11.
- `censo_animales.sql` es el dump inicial.
- Migraciones idempotentes en `db/migrations/NN_*.sql` que se aplican en orden.

Tareas en background

- **Scheduler APScheduler** dentro de Flask:
- Backup diario (con retención configurable).
- Recordatorios de seguros y vacunas.
- Limpieza de logs antiguos sin errores.
- **Tabla de auditoría:** cada acción relevante escribe una línea JSON en

`logs/auditoria.jsonl`.

Flujo de una petición típica

1. El navegador carga `/` → nginx devuelve `web/index.html`. 2. JS hace `fetch("/api/animales")` → nginx hace proxy → Flask consulta MariaDB → JSON. 3. Si la petición es de admin, el cliente envía la cookie/cabecera de sesión y nginx añade `X-Admin-Access: true` desde el vhost 8181. La API la verifica antes de exponer endpoints sensibles.

Roles

Rol	Origen del flag	Permisos
Anónimo	Sin token	Login únicamente
Empleado	Token de EMPLEADOS	CRUD de propietarios y animales

Rol	Origen del flag	Permisos
Policía	Token de POLICIA_USUARIOS	Búsqueda por chip + registrar incidencias
Admin	X-Admin-Access: true + token	Todo lo anterior + auditoría, backups, etc.

Despliegues soportados

- **Docker Compose** (recomendado): `docker compose up -d --build`.
- **Local sin Docker**: requiere MariaDB instalado, `python run_local.py`.

Ver `docs/DESPLIEGUE.md` para detalles.